SmartCampus AI Control Guide

(Domain-Based Architecture + Safe Refactor Mode)

---

1. THE CORE RULE (MOST IMPORTANT)

Always instruct your AI like this:

□ “DO NOT rewrite the system. Only extend and refactor safely.”

---

2. GLOBAL MASTER PROMPT (Use This FIRST)

Paste this into your AI (Replit Agent / ChatGPT) before doing anything:

MASTER SYSTEM INSTRUCTION

```
You are working on a production-level School Management System (SmartCampus).

STRICT RULES:
1. Do NOT break existing functionality
2. Do NOT remove or rewrite existing working code
3. Maintain all existing API endpoints and responses
4. Refactor incrementally using a domain-based architecture
5. Follow Clean Architecture principles:
   - domain (business logic only)
   - application (use cases)
   - infrastructure (database, external services)
   - presentation (API layer)

6. Each change must:
   - be minimal
   - be testable
   - not affect unrelated modules

7. Prefer adding new layers instead of modifying old ones

8. When unsure:
   - ask for clarification instead of guessing

OUTPUT RULES:
- First explain the plan
- Then generate code in small steps
- Never generate the full system at once
```

---

3. PHASE-BASED AI WORKFLOW

You should NEVER ask AI to “build everything.”

PHASE 1: STRUCTURE ONLY (NO LOGIC)

Prompt:

```
Create a domain-based folder structure for the Users domain.

Include:
- domain/
- application/
- infrastructure/
- presentation/

Do NOT implement logic.
Do NOT modify existing files.
```

PHASE 2: INTERFACES (CONTRACT FIRST)

```
Define interfaces for the Users domain:

- StudentRepository
- ParentRepository

Do NOT implement database logic.
Only define method signatures.
```

PHASE 3: USE CASES (CONTROLLED LOGIC)

```
Create a use case:

"Create Student"

Rules:
- Must validate required fields
- Must not duplicate student

Do NOT connect to database yet.
```

PHASE 4: CONNECT TO EXISTING SYSTEM

```
Update the existing Student API view to call the new service layer.

Do NOT change:
- API route
- request/response format

Only replace internal logic.
```

---

4. SAFE REFACTOR PROMPTS (CRITICAL)

Moving Code Safely

```
Move business logic from the current view into a service class.

Rules:
- Do NOT change API behavior
- Do NOT change database models
- Ensure the API still returns the same response
```

Extracting Domain Logic

```
Extract validation logic into the domain layer.

Keep:
- same validation rules
- same error messages
```

Connecting Layers

```
Refactor the code so that:

View → Application Service → Domain → Repository

Do NOT break existing functionality.
```